

Resampling & Markov Chain Monte Carlo

Dr Lily Asquith (Lily, she/her)



Resampling & MCMC

- Monday: Resampling
- Monday: Intro to Markov Chain Monte Carlo.
- Today: Tuning MCMC.
- Today: Marking Scheme for Software Exercise.

The python example I provided this week is [PoissonGamma.py](#). Version 1 of this is on the week 11 canvas page.

It includes a function called `sampler` which is an implementation of the very simple Metropolis Hastings algorithm for generating a sampling distribution for an unknown parameter.

Lily's Metropolis Hastings for Poisson Gamma

(sampler)

The sampler is building an array of values. The values are estimates for the true Poisson μ parameter underlying the data.

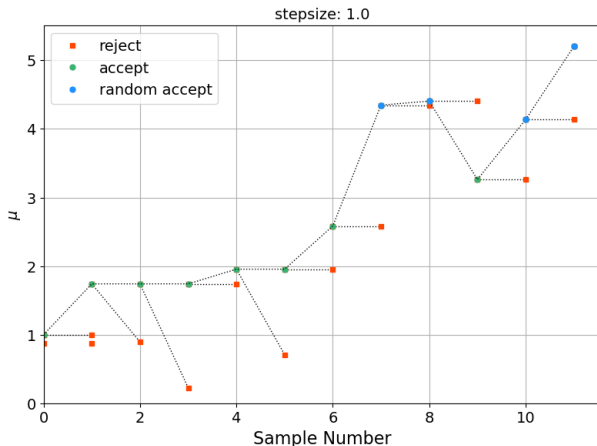
Set a starting value a_0 for the parameter μ . Then begin a `for` loop:

- Use random values from a normal distribution to smear a_0 , creating an alternative, a_1 .
- Create two Poisson distributions, with the parameter μ set to each of these choices a_0 , a_1 .
- Calculate the Log Likelihood of the data in each case: LL_0 and LL_1 .
- Create a single Gamma distribution, with our prior guess for the parameters a and b .
- Calculate two Prior Probabilities for the two suggested parameters a_0 and a_1 : $Prior_0$ and $Prior_1$.
- Construct two Posteriors: $post_0 = LL_0 * Prior_0$ and $post_1 = LL_1 * Prior_1$.
- If the ratio $post_1/post_0 > 1$, accept a_1 . If not, randomly choose whether to accept a_1 or to stick with a_0 .
- Put either a_0 or a_1 in our box of samples.

Then we go to the next iteration of the loop.

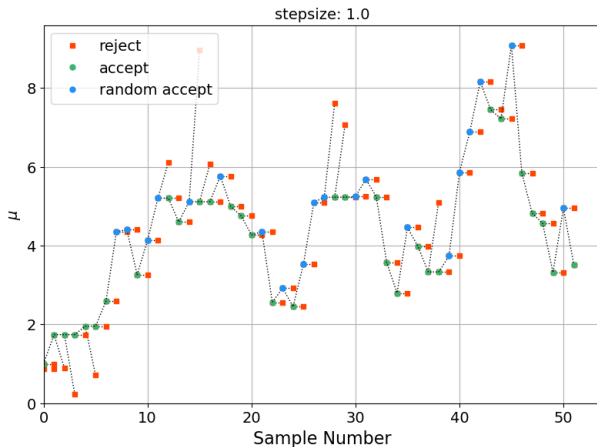
The Algorithm

Looking at the Trace Plots can help us to understand what the algorithm is doing (true $\mu = 4$ for these examples).



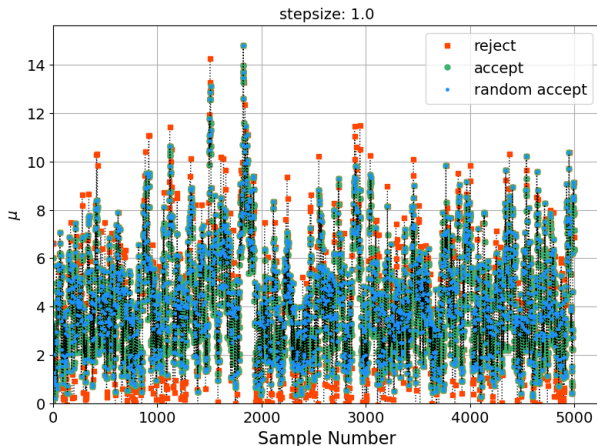
The Algorithm

Looking at the Trace Plots can help us to understand what the algorithm is doing (true $\mu = 4$ for these examples).



The Algorithm

Looking at the Trace Plots can help us to understand what the algorithm is doing (true $\mu = 4$ for these examples).



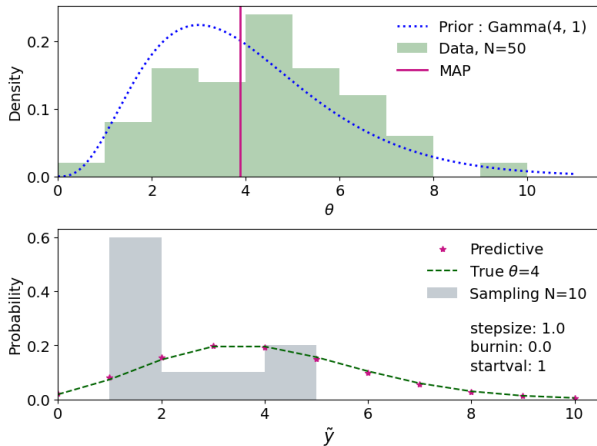
Tuning

The parameters for the 'by-hand' python example of the Metropolis Hastings algorithm (Poisson Gamma) I have provided are:

```
# SAMPLING HYPERPARAMETERS
s_N=5_000      # Number of samples. More is more.
s_guess=-1     # Starting value (-1 means choose randomly from data range)
s_stepsize=5   # Max amount of smearing from the starting value
s_burnin=0.1   # Fraction of samples to use for burning in the sampler
```

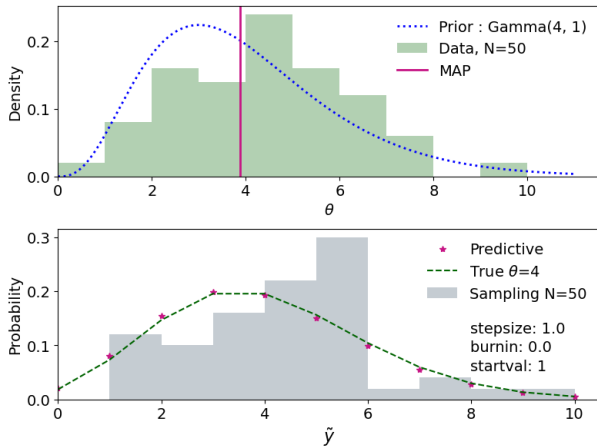
Tuning: Number of Samples

`s_N=10` *# Number of samples. More is more.*



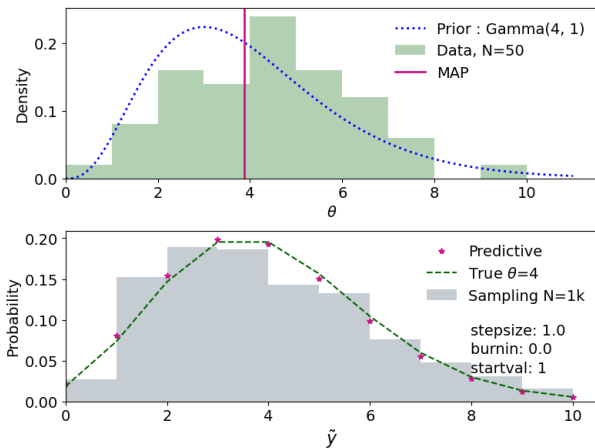
Tuning: Number of Samples

`s_N=50` *# Number of samples. More is more.*



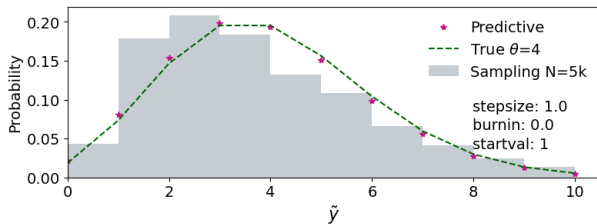
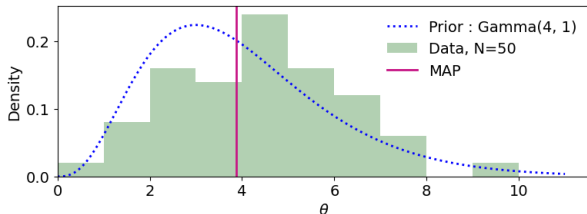
Tuning: Number of Samples

`s_N=1000` *# Number of samples. More is more.*



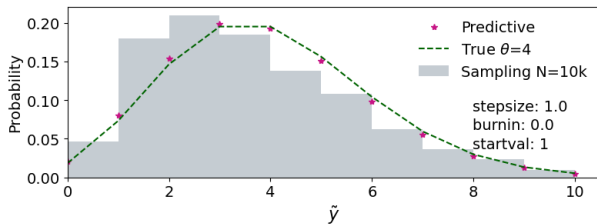
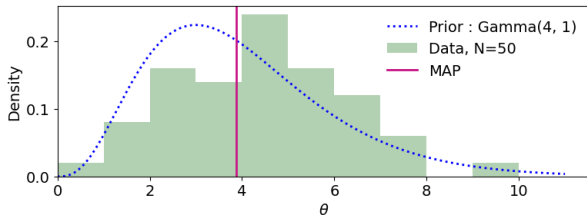
Tuning: Number of Samples

`s_N=5_000` *# Number of samples. More is more.*



Tuning: Number of Samples

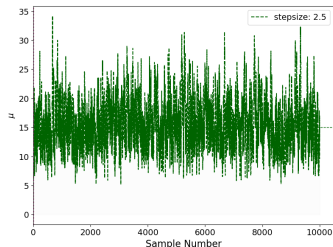
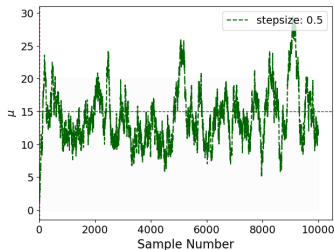
`s_N=10_000` *# Number of samples. More is more.*



Tuning: Step Size

```
s_guess=-1      # Starting value (-1 means choose randomly from data range)
s_stepsize=0.5   # Max amount of smearing from the starting value
```

We can check on our choice of step size by looking at a Trace Plot:

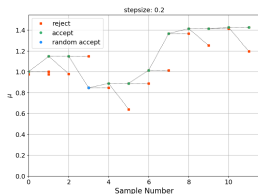


The right hand plot with the larger step size is the better choice here because it is not hyperfocusing on lots of detailed fluctuations in probability, and is covering the true parameter value well.

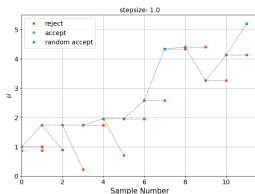
Tuning step-size with Trace Plots

Looking at the first few (ten) samples gives us insight into individual decisions.

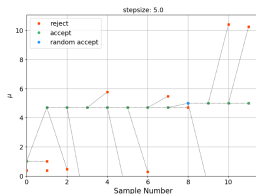
step-size= 0.2



step-size= 1.0



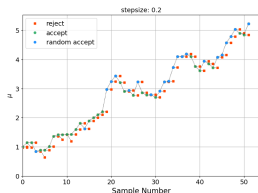
step-size= 5.0



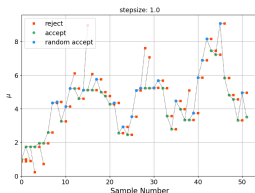
Tuning step-size with Trace Plots

Looking at the first fifty samples gives us insight into how the decisions progress.

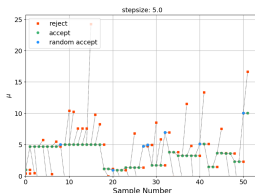
step-size= 0.2



step-size= 1.0



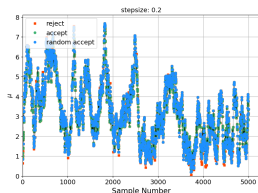
step-size= 5.0



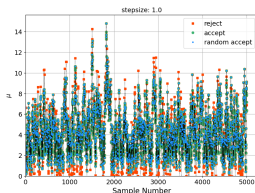
Tuning step-size with Trace Plots

Looking at a large number of samples gives us insight into the big picture (end result).

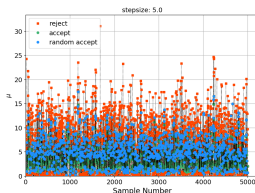
step-size= 0.2



step-size= 1.0



step-size= 5.0

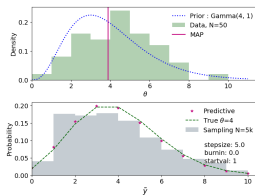
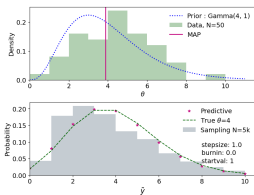
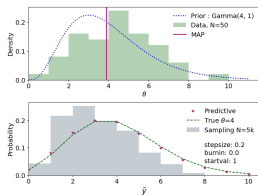


Step-size and Sampling Distribution

step-size= 0.2

step-size= 1.0

step-size= 5.0



Note that the Poisson μ (aka λ) is a discrete RV - the expected number of events in a time period. An integer. Step-size effects will be much more noticeable for a continuous (eg Normal) RV.

Tuning: Burn-in

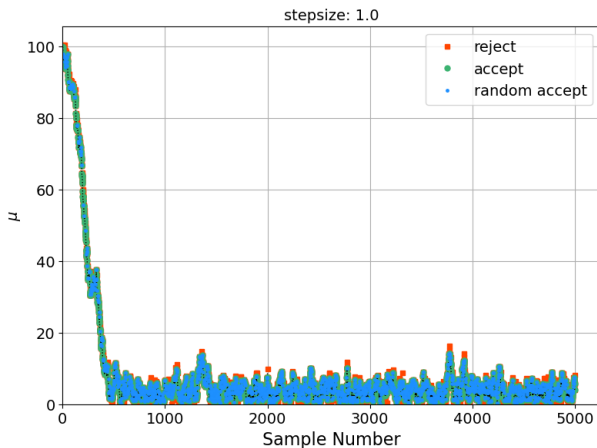
The first steps of a Markov Chain are susceptible to being problematic, because we start at some random point for which there may be tiny probabilities for all values tested, but because we only care about the ratio of probabilities between this one and the last one, we will be accepting all sorts of trash.

You can drop samples from early steps using the burn-in hyperparameter:

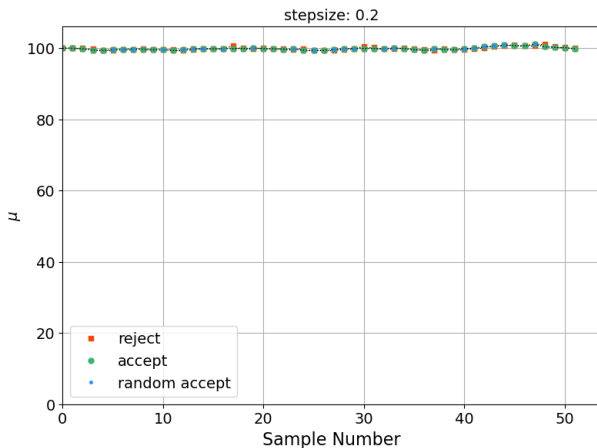
```
s_burnin=0.1    # Fraction of samples to use for burning in the sampler
```

Tuning: Burn-in

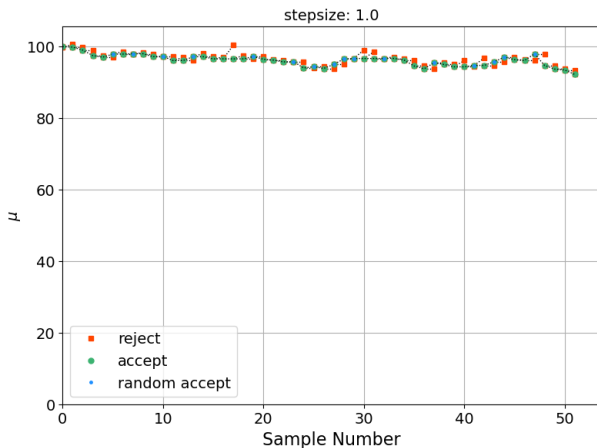
```
s_guess=100    # Guess at starting value is very far from the true parameter value  
s_burnin=0.0    # Fraction of samples to use for burning in the sampler
```



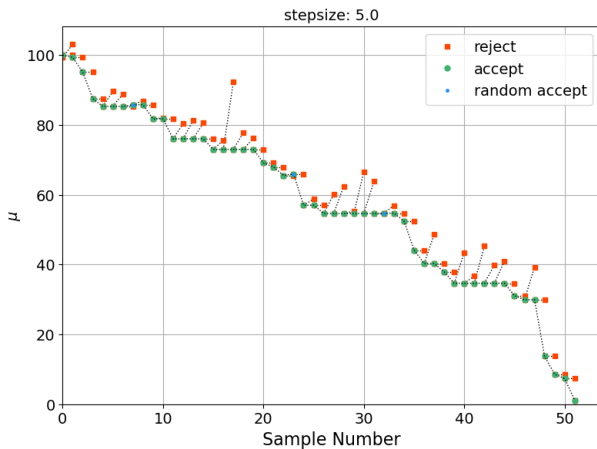
Tuning: Burn-in, Guess, and Step size



Tuning: Burn-in, Guess, and Step size



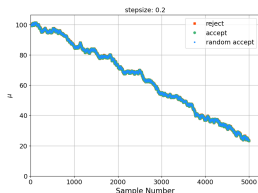
Tuning: Burn-in, Guess, and Step size



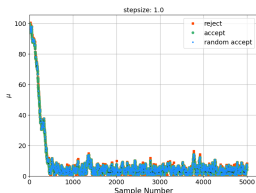
Tuning: Burn-in, Guess, and Step size

These tuning hyperparameters should be considered together over large number of samples: dark arts!

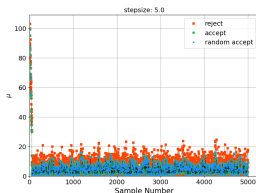
step-size= 0.2



step-size= 1.0



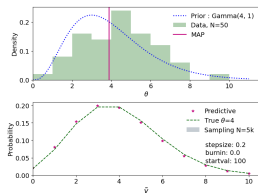
step-size= 5.0



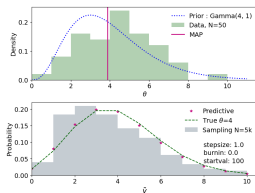
Tuning: Burn-in, Guess, and Step size

These tuning hyperparameters should be considered together over large number of samples: dark arts!

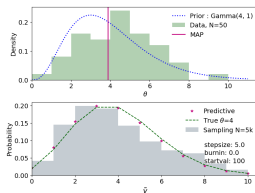
step-size= 0.2



step-size= 1.0

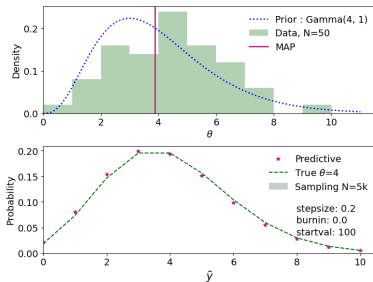
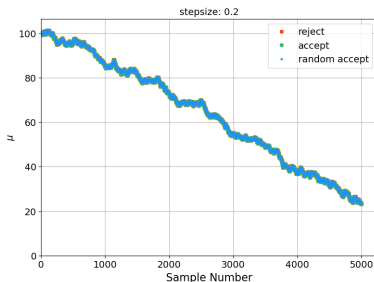


step-size= 5.0



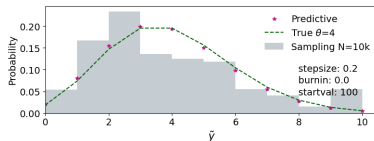
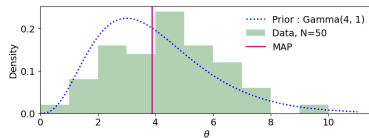
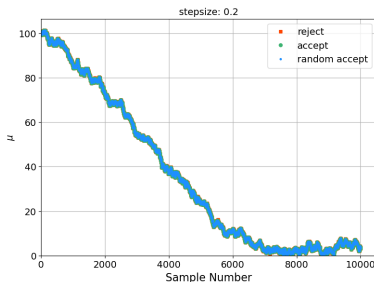
Tuning: Burn-in, Guess, and Step size

Even with a poor starting value and an inappropriate step size, given enough samples the algorithm should eventually start to converge.



Tuning: Burn-in, Guess, and Step size

Even with a poor starting value and an inappropriate step size, given enough samples the algorithm should eventually start to converge.



The unforgiving part is the math

The part that you must get right is the math.

Take the log of the likelihood of the data- otherwise you will have problems with numbers that are too vast or tiny for the computer to handle.

You will be calculating the posterior for individual numerical guesses at the parameter value. For every value you must recalculate both the log likelihood and the prior. The product of these is the posterior.

All over the internet people say the algorithm compares likelihoods. They mean posteriors!

For the software exercise, someone did the math for us to get the parameter defs we need. We can still make mistakes, so debug carefully to sanity check what you are doing over a few samples.

Contacting me over the break

Please do! I will be working until end of next week (Friday 19 Dec)

From 20-28 December I fully expect my daughter to put my laptop under lock and key.

You can email me whenever you like, I may take a few days to respond.

All questions are good questions.

I expect you to have looked at the lecture notes, workshops, python examples. What you need is all in there.

Use the internet! Carefully! Enormous amounts of trash out there.

Thanks For Your Time

Fin.